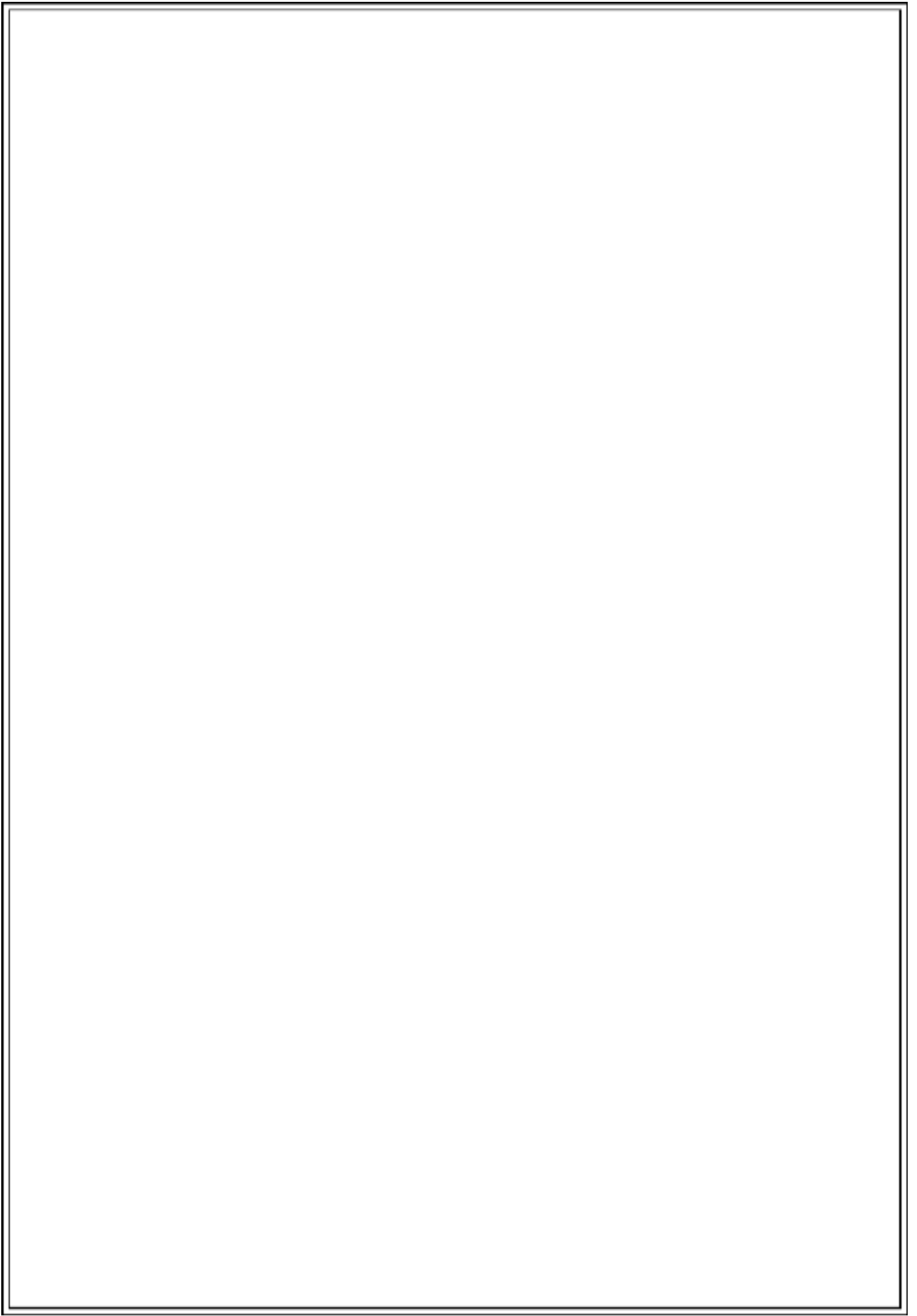




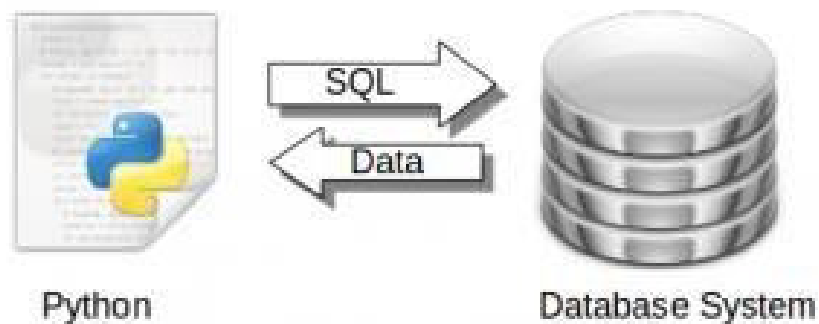
HARDWARE AND SOFTWARE REQUIRED

HARDWARE

1. PC
2. MOBILE PHONE

SOFTWARE

- PYTHON (latest version)
- MYSQL
- PYTHON CONNECTOR



CRUISE

The following are the main advantages:

- High Speed: It is the fast speed means of transport. ...
- Minimum Cost: ...
- Strategic Importance: ...
- Easy transport of costly and light goods: ...
- Free from physical barriers: ...
- Useful for Agriculture: ...
- Useful in natural calamities:

CRUISE MANAGEMENT SYSTEM

A Cruise Management System (CMS) is an essential component of modern cruise ship operations under marine management. A CMS is a computerized system designed to automate and manage various operational tasks involved in cruise travel, thereby reducing manual workload and improving efficiency. It helps cruise operators manage passenger bookings, cruise schedules, cabin allocation, food services, luggage handling, and overall voyage planning.

The primary function of a Cruise Management System is the effective management of cruise operations from departure to arrival. The system maintains accurate records of cruise routes, source and destination ports, sailing schedules, passenger details, and onboard services. By using computerized databases and management software, the CMS ensures smooth coordination between different departments of the cruise line.

From the ship's control and management offices, the CMS is accessed through computer terminals and software interfaces that allow administrators and staff to update, retrieve, and manage data efficiently. The system displays cruise details, passenger records, and booking information, helping in decision-making and service management.

Modern Cruise Management Systems are used in large luxury cruise ships as well as smaller passenger vessels. Over time, CMS has evolved with improved capabilities, flexibility, and user-friendly interfaces. Despite variations in size and functionality, all Cruise Management Systems share the common goal of enhancing operational efficiency, ensuring passenger satisfaction, and supporting effective marine transportation management.

NEED OF CMS

1. Minimized documentation and no duplication of records.
2. Reduced paper work.
3. Improved patient care
4. Better Administration Control
5. Faster information flow between various departments
6. Smart Revenue Management
7. Effective billing of various services
8. Exact stock information

#CODE :

```
#FLIGHT MANAGEMENT SYSTEM
#USING PYTHON+MYSQL
#CREATED BY GIRIDHARAN A.S
```

```
import mysql.connector #_____mysql connector package
```

```
obj=mysql.connector.connect(host="localhost",user="root",passwd="admin")
#here obj is connection object
```

#CREATING DATABASE & TABLE

```
mycursor=obj.cursor()
```

```
#_____cursor is used to row by row processing of record in
the resultset
```

```
mycursor.execute("create database if not exists cruise")
#_____mycursor is cursor object
```

#CREATING DATABASE

```
Mycursor.execute("create database if not exists cruise")
mycursor.execute("use cruise")
```

#CREATING TABLE FOR ODER FOOD

```
mycursor.execute("create table if not exists food_items(sl_no int(4)
auto_increment primary key,food_name varchar(40)not null,price int(4)
not null)")
mycursor.execute("insert into food_items
values({},'{}',{})".format('null','pepsi',150))
mycursor.execute("insert into food_items
values({},'{}',{})".format('null','coffee',70))
mycursor.execute("insert into food_items
values({},'{}',{})".format('null','tea',50))
mycursor.execute("insert into food_items
values({},'{}',{})".format('null','water',60))
mycursor.execute("insert into food_items
values({},'{}',{})".format('null','milk shake',80))
mycursor.execute("insert into food_items
values({},'{}',{})".format('null','chicken burger',160))
```

```

mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', 'cheese pizza', 70))
mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', 'chicken biryani', 300))
mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', 'plane rice', 80))
mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', 'aloo paratha', 120))
mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', 'roti sabji', 100))
mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', 'omelette', 50))

```

#CREATING TABLE FOR LUGGAGE ENTRY

```

mycursor.execute("create table if not exists luggage(luggage_id int(4)
auto_increment primary key,weight int(3)not null,price int(4) not
null)")

```

#CREATING TABLE FOR CUSTOMER DETAILS

```

mycursor.execute("create table if not exists cust_details(cust_id int(4)
auto_increment primary key,cust_name varchar(40)not null,cont_no
bigint(10) not null)")

```

#CREATING TABLE FOR CUSTOMER'S CRUISE DETAILS

```

mycursor.execute("create table if not exists crusie_details(cus_id
int(4),cus_name varchar(40)not null,_id)")

```

```

obj.commit()

```

#TO ENTER THE DETAILS OF LUGGAGE

```

def luggage():
    print("what do you want to
do?") print("1. add luggage")
    print("2. delete luggage")
    x=int(input("enter your choice: "))
    if x==1:
        lname=input("enter luggage type: ")
        mycursor.execute("insert into luggage
values({}, '{}'.format('null', lname))
    elif x==2:
        lid=int(input("enter your luggage id: "))
        mycursor.execute("delete from luggage where
luggage_id={}".format(lid))

    else:
        print(" ***** PLEASE ENTER A VALID OPTION
***** ")
        food()
    obj.commit()

```

#TO UPDATE THE INFORMATION OF FOOD DETAILS

```
def food():
    print("what do you want to do?")
    print("1. add new items")
    print("2. update price")
    print("3. delete items")
    x=int(input("enter your choice: "))
    if x==1:
        fname=input("enter food name: ")
        fprice=int(input("enter food price: "))
        mycursor.execute("insert into food_items
values({}, '{}', {})".format('null', fname, fprice))
    elif x==2:
        fid=int(input("enter food id: "))
        fprice=int(input("enter new price: "))
        mycursor.execute("update food_items set price={} where
food_id={} ".format(fid, fprice))
    elif x==3:
        fid=int(input("enter food id: "))
        mycursor.execute("delete from food_items where where
food_id={} ".format(fid))
    else:
        print(" ***** PLEASE ENTER A VALID OPTION
***** ")
        food()
    obj.commit()
```

#TO UPDATE THE INFORMATION OF CLASSTYPE

```
def classtype():
    print("what do you wat to do? ")
    print("1. change the classtype name")
    print("2. change the price of
classtype") x=int(input("enter your
choice: "))
    if x==1:
        oname=input("enter old name: ")
        nname=input("enter new name: ")
        mycursor.execute("update classtype set
'{}'='{}' ".format(oname, nname))

def fooditems():
    print(" ")
    print(" ")

    print("          THE AVAILABLE FOODS ARE:          ")
    print(" ")
    print(" ")
    mycursor.execute("select * from food_items")
```

```

x=mycursor.fetchall()
for i in x:
    print(" FOOD ID: ",i[0])
    print(" FOOD Name: ",i[1])
    print(" PRICE: ",i[2])

print("_____")
_____")

user()

```

#Admin Interface after verifying password

```

def admin1():
    print("***** WHAT'S YOUR TODAYS GOAL? *****")
    print("1. update details")
    print("2. show details")
    print("3. job approval")
    x=int(input("select your choice: "))
    while True:
        if x==1:
            print("1. classtype")
            print("2. food")
            print("3. luggage")
            x1=int(input("enter your choice"))
            if x1==1:
                classtype()
            elif x1==2:
                food()
            elif x1==3:
                luggage()
            else:
                print(" ***** PLEASE ENTER A VALID
OPTION ***** ")
                admin1()

        elif x==2:
            print("1. classtype")
            print("2. food")
            print("3. luggage")
            print("4. records")
            y=int(input("from which table: "))
            if y==1:
                mycursor.execute("select * from classtype")
            else:
                mycursor.execute("select * from customer_details")
                z=mycursor.fetchall()
                for i in z:
                    print(i)
                print("***** THESE ABOVE PEOPLE HAVE BOOKED
TICKET *****")
                break

```


#Admin Interface

```
def admin():
    while True:
        sec=input("enter the password: ")
        if sec=="admin":
            admin1()
        else:
            print("*****YOUR PASSWORD IS INCORRECT*****")
            print("*****PLEASE TRY AGAIN*****")
            admin()
    break
```

#TO SEE THE RECORDS OF THE CUSTOMER

```
def record():
    cid=int(input("enter your customer id: "))
    mycursor.execute(" select * from customer_details where
cus_id={}".format(cid))
    d=mycursor.fetchall()

    print("YOUR DETAILS ARE HERE          ")
    print("customer id: ",d[0])
    print("name: ",d[1])
    print("mobile number: ",d[2])
    print("cruise id: ",d[3])
    print("cruise name",d[4])
    print("classtype: ",d[5])
    print("departure port",d[6])
    print("destination port",d[7])
    print("sailing day: ",d[8])
    print("boarding time: ",d[9])
    print("price of ticket: ",d[10])
    print("date of booking ticket:
",d[11])
```

#TO BOOK THE TICKETS

```
def ticketbooking():
    cname=input("enter your name: ")
    cmob=int(input("enter your mobileno: "))
    if cmob==0000000000:
        print(" MOBILE NUMBER CANT BE NULL ")
        ticketbooking()
    cid=int(input("enter cruise id: "))
    ccl=input("enter your class: ")
    cname=input("enter your cruise name")
    dept=input("enter departure port: ")
```

```

dest=input("enter destination port: ")
cday=input("enter sailing day: ")
ctime=input("enter boarding time: ")
cprice=input("enter ticket rate: ")
mycursor.execute("insert into customer_details
values({},'{}',{},{},'{}','{}','{}','{}','{}','{}').format('null',cnam
e,cmob,cid,cname,ccl,dept,dest,cday,ctime,"curdate()")

obj.commit()

```

#TO SEE THE AVAILABLE CRUISE

```

def cruiseavailable():
    print(" ")
    print(" ")

    print("                THE AVAILABLE CRUISE ARE:                ")
    print(" ")
    print(" ")
    mycursor.execute("select * from cruise_details")
    x=mycursor.fetchall()
    for i in x:
        print(" ")
        print(" Cruise ID: ",i[0])
        print(" Cruise Name: ",i[1])
        print(" departure port: ",i[2])
        print(" Destination port: ",i[3])
        print(" Sailing Day: ",i[4])
        print(" Boarding time : ",i[5])
        print(" bussiness : ",i[6])
        print(" middle : ",i[7])
        print(" economic : ",i[8])

    print("_____")
    print("_____")

    user()

```

#USER INTERFACE

```

def user():
    while True:
        print("***** MAY I HELP YOU? *****")
        print("1. cruise details")
        print("2. food details")
        print("3. book ticket")
        print("4. my details")
        print("5. exit")
        x=int(input("enter your choice: "))
        if x==1:
            cruiseavailable()
        elif x==2:
            fooditems()
        elif x==3:

```

```

        ticketbooking()
    elif x==4:
        records()
    else:
        print("***** PLEASE CHOOSE A CORRECT OPTION
*****")
        user(
    ) break

```

```

print("***** WELCOME TO LAMNIO CRUISE
*****")
print("***** MAKE YOUR JOURNEY SUCCESS WITH US!
*****")
print(" ")
print(" ")
print(" ")
print(" ")
print(" ")

```

#Main Interface

```

def menu1():
    print("***** YOUR DESIGNATION PORT?
*****") print("1. admin")
    print("2. user")
    print("3. exit")
    x=int(input("choose a option: "))
    while True:
        if x==1:
            admin()
        elif x==2:
            user(
        ) else:
            print("*****PLEASE CHOOSE A CORRECT
OPTION*****")
            menu1()
        break

```

```

menu1()

```

#mysql

- Tables in database cruise:

```
mysql> use airlines;
Database changed
mysql> show tables;
+-----+
| Tables_in_airlines |
+-----+
| class_details      |
| customer_details  |
| flight_details     |
| food_items         |
| luggage            |
+-----+
5 rows in set (0.04 sec)
```

- Description and datas in table class_details:

```
mysql> desc class_details;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| class_id | int | NO | PRI | NULL | auto_increment |
| classtype | varchar(40) | YES | | NULL | |
| price | int | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql> select * from class_details;
+-----+-----+-----+
| class_id | classtype | price |
+-----+-----+-----+
| 101 | economic | 2000 |
| 102 | middle | 4000 |
| 103 | bussiness | 6000 |
+-----+-----+-----+
```

- Description and datas in table customer_details:

```
mysql> desc customer_details;
```

Field	Type	Null	Key	Default	Extra
cus_id	int	NO	PRI	NULL	auto_increment
cus_name	varchar(40)	YES		NULL	
mob_no	bigint	YES		NULL	
fl_id	int	YES		NULL	
fl_name	varchar(40)	YES		NULL	
class	varchar(15)	YES		NULL	
dept	varchar(20)	YES		NULL	
dest	varchar(20)	YES		NULL	
day	varchar(15)	YES		NULL	
f_time	time	YES		NULL	
price	int	YES		NULL	

11 rows in set (0.01 sec)

```
mysql> select * from customer_details;
```

cus_id	cus_name	mob_no	fl_id	fl_name	class	dept	dest	day	f_time	price
10001	hibu myodi	1234567891	2001	spice-jet	bussiness	delhi	mumbai	sunday	04:00:00	14000
10002	rejoice basumatary	9876543212	2002	air india	middle	guwahati	kolkata	monday	06:00:00	14000
10003	Lungsom Lamnio	8413050187	2003	go first	economic	haryana	goa	tuesday	01:00:00	13000
10004	nangbia teji	8765987612	2004	vistara	bussiness	varoda	chennai	wednesday	15:00:00	20000
10005	igon lona	9876587612	2005	indigo	middle	hollongi	punjab	thursday	23:00:00	22000
10006	lakhi mili	1122334455	2006	alliance air	economic	guwahati	madras	friday	17:00:00	24000
10007	ankit das	1122556677	2007	flybig	bussiness	mumbai	sikkim	saturday	10:00:00	45000

- Description and datas in table _details:

```
mysql> desc flight_details;
```

Field	Type	Null	Key	Default	Extra
flight_id	int	NO	PRI	NULL	auto_increment
flight_name	varchar(40)	YES		NULL	
departure	varchar(40)	YES		NULL	
destination	varchar(40)	YES		NULL	
day	varchar(40)	YES		NULL	
f_time	time	YES		NULL	
bussiness	int	YES		NULL	
middle	int	YES		NULL	
economic	int	YES		NULL	

9 rows in set (0.00 sec)

```
mysql> select * from flight_details;
```

flight_id	flight_name	departure	destination	day	f_time	bussiness	middle	economic
2001	spice-jet	delhi	mumbai	sunday	04:00:00	14000	15000	17000
2002	air-india	guwahati	kolkata	monday	06:00:00	13000	14000	16000
2003	go-first	haryana	goa	tuesday	01:00:00	10000	11000	13000
2004	vistara	varoda	chennai	wednesday	15:00:00	20000	21000	23000
2005	indigo	hollongi	punjab	thursday	23:00:00	25000	26000	28000
2006	alliance air	guwahati	madras	friday	17:00:00	21000	22000	24000
2007	flybig	mumbai	los angeles	saturday	10:00:00	45000	46000	48000

7 rows in set (0.03 sec)

Description and datas in table food_items:

```
mysql> desc food_items;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| sl_no | int  | NO   | PRI | NULL    | auto_increment |
| food_name | varchar(40) | NO | | NULL    |
| price | int  | NO   | | NULL    |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> select * from food_items;
+-----+-----+-----+
| sl_no | food_name | price |
+-----+-----+-----+
| 1 | pepsi | 150 |
| 2 | coffee | 70 |
| 3 | tea | 50 |
| 4 | water | 60 |
| 5 | milk shake | 80 |
| 6 | chicken burger | 160 |
| 7 | cheese pizza | 70 |
| 8 | chicken biryani | 300 |
| 9 | plane rice | 80 |
| 10 | aloo paratha | 120 |
| 11 | roti sabji | 100 |
| 12 | omelette | 50 |
+-----+-----+-----+
12 rows in set (0.01 sec)
```

- Description and datas in table luggage:

```
mysql> desc luggage;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| luggage_id | int | NO | PRI | NULL | auto_increment |
| weight | int | NO | | NULL |
| price | int | NO | | NULL |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.03 sec)

mysql> select * from luggage;
+-----+-----+-----+
| luggage_id | weight | price |
+-----+-----+-----+
| 1 | 25 | 1200 |
| 2 | 17 | 950 |
| 3 | 30 | 1500 |
| 4 | 40 | 2000 |
+-----+-----+-----+
4 rows in set (0.03 sec)
```

BIBLIOGRAPHY

- CLASS XI AND XII NCERT TEX